A Comparative Study of Shared-Memory, GPU, and Distributed Parallelisation Strategies for 2D Image Convolution

Project Synopsis — Parallel Computing Architecture and Programming (PCAP)

Saatvik Das, Saket Rama, Zaid Khan
Department of Computer Science and Engineering
Manipal Institute of Technology, Bengaluru

Guide: Prof. Anil Kumar
Semester V

***Abstract*—**Two-dimensional convolution is the computational core of classical image processing and of the convolutional layers that dominate modern computer vision. It is also a textbook memory-bound stencil computation: for a K×K filter, a naive implementation reads every input pixel K^2 times from memory while performing only $2K^2$ floating-point operations per output pixel, giving an arithmetic intensity of roughly 0.5 FLOP/byte. Consequently, raw parallelism alone does not deliver proportional speedup — performance is governed by how well the memory hierarchy is exploited. This project implements a single, well-defined algorithm (2D convolution with an arbitrary odd-sized filter over large images) across four progressively parallel execution models: a serial C++ baseline, an OpenMP multi-threaded version, a naive CUDA version using only global memory, and an optimised CUDA version that places filter coefficients in constant memory and stages input tiles with halo regions in shared memory. An optional MPI layer distributes horizontal image strips across ranks to demonstrate distributed-memory decomposition. All four implementations are validated against the serial reference and benchmarked on 4K and 8K images with filter sizes swept from 3×3 to 15×15. The study quantifies OpenMP scaling and its bandwidth-imposed plateau, the effect of CUDA block dimensions on runtime, the reuse-driven advantage of tiling, and the honest cost of host–device transfers. Measured serial fractions are fed into Amdahl's law to compare predicted and observed speedup ceilings. The deliverable is a reproducible benchmark suite with a plotting script and an analytical report.

***Index Terms*—**2D convolution, OpenMP, CUDA, shared memory, constant memory, MPI, Amdahl's law, roofline model, GPU acceleration, memory bandwidth, parallel computing.

I. Introduction

For roughly four decades, application performance improved automatically as transistor density and clock frequency rose together. That arrangement ended in the mid-2000s: Dennard scaling broke down, power density became the binding constraint, and clock frequencies plateaued near 3–5 GHz. Vendors responded by scaling *outward* rather than *upward* — multi-core CPUs, wide SIMD units, many-core GPUs, and clustered nodes. The consequence for software is that performance is no longer inherited; it must be explicitly engineered. This shift is precisely what parallel computing as a discipline exists to address, and it is why a single algorithm can differ by two or three orders of magnitude in runtime depending only on how it is mapped to hardware.

Within this landscape, image convolution is an unusually good subject of study. It is embarrassingly parallel at the output level — every output pixel is independent — so it exposes the parallel programming models cleanly without algorithmic distractions. At the same time it is *not* trivially fast, because its data access pattern is a sliding stencil with heavy overlap between neighbouring output elements. That overlap is simultaneously the source of the problem (redundant global memory traffic) and the source of the solution (data reuse through cache blocking and shared-memory tiling). Convolution therefore forces a programmer to confront the central lesson of modern parallel programming: on contemporary hardware, the scarce resource is memory bandwidth, not arithmetic throughput.

The workload is also of continuing practical importance. Convolution underlies Gaussian blurring, sharpening, edge detection (Sobel, Laplacian, Canny), denoising, and unsharp masking in image and video pipelines; it is the dominant cost in the forward and backward passes of convolutional neural networks; and it appears in medical imaging reconstruction, satellite and remote-sensing pre-processing, and real-time video filtering. Image resolutions have grown faster than single-core performance — 4K (3840×2160) is now a consumer baseline and 8K is common in professional capture — so a filter that was cheap on a 512×512 image becomes a real bottleneck at 8.3 or 33.2 megapixels, especially at larger kernel radii.

Highly optimised implementations already exist (cuDNN, OpenCV, oneDNN, FFT- and Winograd-based methods). However, these are delivered as tuned black boxes: a user calls a library function and observes a number, without visibility into *which* optimisation produced the speedup, or how the trade-offs shift as filter size, image size, and launch configuration change. From an engineering and pedagogical standpoint, there is value in building the same algorithm four times, under identical measurement conditions, and attributing each increment of performance to a specific, named hardware mechanism.

II. Problem Statement

Existing high-performance convolution implementations are distributed as opaque, pre-tuned library primitives that report end-to-end timings without exposing which architectural mechanism — thread-level parallelism, cache blocking, constant-memory broadcast, or shared-memory reuse — is responsible for the observed gain. Published comparisons are

typically confined to a single programming model (OpenMP *or* CUDA), use small images and 3×3 filters that fit comfortably in cache, and report only kernel time while omitting host–device transfer cost, thereby overstating realistic GPU advantage. Furthermore, the theoretical scaling limits predicted by Amdahl's law are rarely validated against measured serial fractions on the same workload. There is consequently no compact, reproducible study that implements one identical algorithm across serial, shared-memory, GPU-naive, GPU-optimised, and distributed-memory models, sweeps both filter size and launch configuration, and attributes performance changes to specific levels of the memory hierarchy.

III. Literature Survey

Note: verify each citation against the original source and adjust to your institution's required format before submission.

[1] Amdahl (1967) established that the achievable speedup of a program is bounded by its non-parallelisable fraction, giving the ceiling $S(N) = 1 / (f + (1–f)/N)$. This remains the standard first-order model for scaling limits. Its limitation is that it assumes a *fixed* problem size and a serial fraction independent of thread count; it does not account for parallel overheads (thread spawn, synchronisation, PCIe transfer) or for memory-bandwidth saturation, both of which cause real systems to plateau well below the Amdahl ceiling — a discrepancy this project measures directly.

[2] Datta et al. (2008) performed a landmark auto-tuning study of stencil computations across multicore CPUs and early GPUs, demonstrating conclusively that stencil kernels are bandwidth-bound and that cache/register blocking, not additional cores, drives most of the gain. However, the work targeted 3D finite-difference stencils on hardware generations that predate unified L2 caches and modern CUDA memory hierarchies, and it did not isolate the contribution of constant memory for filter coefficients.

[3] Micikevicius (2009) introduced the now-canonical shared-memory tiling pattern for 3D finite-difference stencils on CUDA, including explicit halo (ghost-cell) loading and `__syncthreads()` separation of the load and compute phases, reporting order-of-magnitude gains over global-memory-only kernels. Its scope was restricted to a fixed low-order stencil on Tesla-generation hardware, and it offered no cross-paradigm comparison against a CPU or OpenMP baseline, so the relative merit of GPU versus well-threaded CPU execution was left unquantified.

[4] Chetlur et al. (2014) presented cuDNN, a library of hand-tuned GPU primitives for deep learning that reformulates convolution as `im2col` followed by GEMM to exploit highly optimised matrix-multiply kernels. While extremely fast, the approach materialises a substantially expanded intermediate matrix, incurring significant memory overhead, and the implementation is closed-source and tuned for the small filters and batched tensor shapes typical of CNN layers rather than for large single-image convolutions with kernels up to 15×15.

[5] Lavin and Gray (2016) applied Winograd minimal-filtering algorithms to convolution, reducing the multiplication count for small filters and achieving significant throughput improvements over direct and FFT-based methods. The technique's benefit is confined to small kernels — it degrades in numerical stability and loses its advantage as filter size grows — and it obscures rather than illuminates the underlying memory-hierarchy behaviour, since the speedup arises from arithmetic restructuring rather than from data locality.

[6] Volkov and Demmel (2008) showed that maximising GPU occupancy is not synonymous with maximising performance, and that register and shared-memory pressure per thread frequently make lower-occupancy configurations faster. This finding directly motivates the block-dimension sweep in this project, though the original work addressed dense linear algebra rather than stencil workloads and predates the shared-memory carve-out configurability of current architectures.

IV. Aim and Objectives

A. Aim

To design, implement, and empirically evaluate four progressively parallelised implementations of 2D image convolution — serial, OpenMP, naive CUDA, and memory-hierarchy-optimised CUDA — in order to quantify the individual performance contribution of each parallelisation and memory-optimisation technique, and to validate the observed scaling behaviour against established theoretical models.

B. Objectives

1) To **design and implement** a correct serial C++ reference implementation of 2D convolution supporting arbitrary odd filter sizes (3×3 to 15×15) and configurable boundary handling, to serve as the correctness oracle and speedup denominator for all subsequent versions.
2) To **develop and tune** an OpenMP implementation employing row-wise `parallel for` decomposition, a `collapse(2)` variant, and `static`, `dynamic`, and `guided` scheduling policies, and to measure strong-scaling speedup and efficiency from 1 to 2× the physical core count.
3) To **implement** a naive CUDA kernel using one thread per output pixel with all operands read from global memory, and to characterise the effect of block dimensions (8×8, 16×16, 32×32) on kernel execution time and achieved occupancy.
4) To **optimise** the CUDA implementation by storing filter coefficients in `__constant__` memory and staging input tiles with halo regions in `__shared__` memory, and to demonstrate the resulting reduction in global-memory traffic through both an analytical arithmetic-intensity argument and profiler-measured DRAM throughput.
5) To **evaluate and compare** all implementations across a sweep of image resolutions (4K, 8K) and filter sizes, reporting kernel-only versus end-to-end timings inclusive of `cudaMemcpy`, and to apply Amdahl's law to the measured serial fraction so as to compare the predicted speedup ceiling with the observed plateau.
6) *(Extension)* To **demonstrate** distributed-memory decomposition via an MPI layer that scatters horizontal image strips with halo exchange across ranks, each rank executing the OpenMP or CUDA kernel locally, with results

gathered at rank 0.

V. Proposed Methodology and System Architecture

A. Algorithm Definition

For an input image I of dimensions H×W and a filter F of size K×K (K odd, radius R = (K–1)/2), the output is given by the following stencil summation:

$$O[y][x] = \sum_{i=-R}^{R} \sum_{j=-R}^{R} I[y+i][x+j] \cdot F[i+R][j+R]$$

Boundary pixels are handled by zero-padding (with clamp-to-edge available as a compile-time alternative). All computation is performed in single-precision `float`. The filter is a normalised Gaussian by default; Sobel and box filters are provided for visual verification of correctness.

B. Implementation Stages

Stage 1 — Serial Baseline (C++). Four nested loops (output rows → output columns → filter rows → filter columns), no vectorisation hints, compiled at `-O2` to avoid the compiler auto-parallelising the reference away. This defines T_{serial}, the denominator for every reported speedup, and produces the reference output buffer against which all other versions are validated.

Stage 2 — OpenMP Implementation. `#pragma omp parallel for` applied over output rows, with the filter loops kept serial inside each thread. Variants compared: row-level parallelism versus `collapse(2)` over rows and columns; scheduling policies `static`, `dynamic` with chunk sizes {1, 8, 32}, and `guided`; and thread counts swept over {1, 2, 4, 6, 8, 12, 16} (up to 2× physical cores, to observe SMT behaviour). Because the workload per row is uniform, `static` is expected to win; the value of the comparison lies in demonstrating that dynamic scheduling's overhead is unrecovered when there is no load imbalance to correct.

Stage 3 — Naive CUDA. One thread per output pixel. Grid dimensions computed as $\text{ceil}(W/B_X) \times \text{ceil}(H/B_Y)$ with boundary guards for non-divisible dimensions. Both the input image and the filter are read from global memory on every access. Host–device transfers use `cudaMemcpy`; kernel time is isolated using CUDA events while total time is measured with host timers spanning H2D transfer, kernel launch, and D2H transfer.

Stage 4 — Optimised CUDA. Two distinct optimisations, introduced separately so their contributions can be attributed independently. **Constant memory:** filter coefficients are copied via `cudaMemcpyToSymbol` into a `__constant__ float d_filter[MAX_K*MAX_K]` array. At K=15 this occupies 900 bytes, comfortably within the 64 KB constant space; all threads in a warp read the same coefficient at each step, the ideal broadcast pattern for the constant cache. **Shared-memory tiling:** each block cooperatively loads a $(T+K-1)^2$ tile — the output tile plus its halo/ghost cells — into `__shared__` memory, issues `__syncthreads()`, then computes entirely from shared memory. Shared-memory footprint at T=32, K=15 is $46^2 \times 4$ B ≈ 8.5 KB per block, within the 48 KB limit but with clear implications for occupancy that will be discussed.

Stage 5 — MPI Layer (optional extension). Rank 0 reads the image and distributes horizontal strips of H/N + 2R rows (the ±R overlap constituting the halo) via `MPI_Scatterv`. Each rank convolves its strip using its local OpenMP or CUDA kernel. Interior results (excluding halo rows) are collected with `MPI_Gatherv`. This is deliberately kept simple; the intent is to demonstrate domain decomposition and ghost-region exchange, not to pursue communication-optimal distribution.

C. System Architecture

Fig. 1 shows the data flow from image ingestion through the five parallel execution paths to the shared correctness-validation and timing-analysis pipeline.

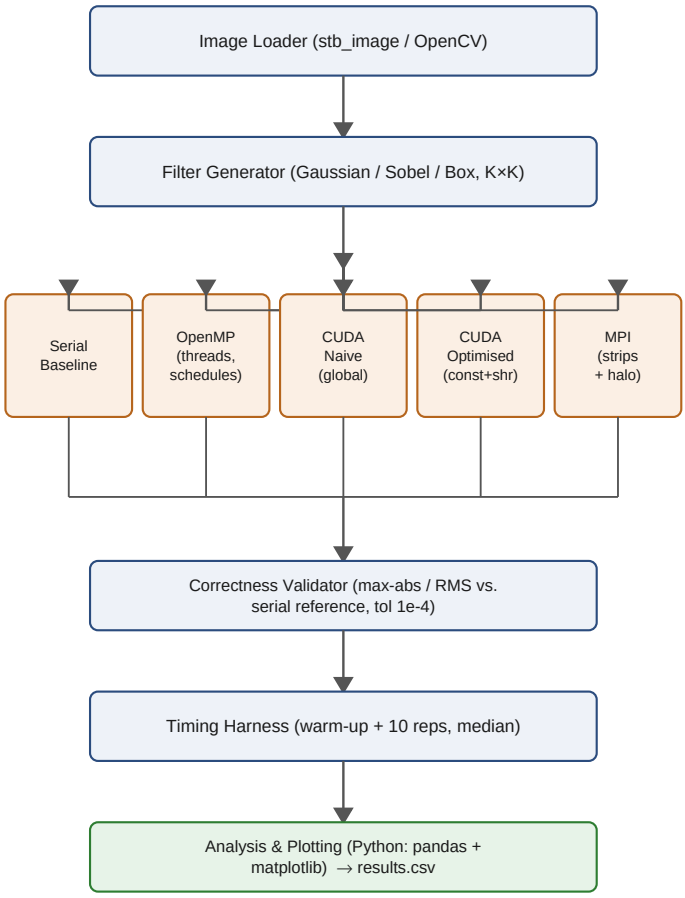

Fig. 1. System architecture and data flow across the five execution paths.

D. Experimental Setup and Measurement Protocol

TABLE I
Experimental Parameter Sweep

Parameter	Values Swept
Image resolution	3840×2160 (4K), 7680×4320 (8K); 4–5 distinct images
Filter size K	3, 5, 7, 9, 11, 13, 15
OpenMP thread count	1, 2, 4, 6, 8, 12, 16
OpenMP schedule	static, dynamic(1/8/32), guided; with and without collapse(2)
CUDA block dimensions	8×8, 16×16, 32×32
CUDA variant	naive, +constant only, +constant +shared (tiled)

Timing methodology. CPU timings use `std::chrono::steady_clock`; GPU kernel timings use `cudaEventRecord` / `cudaEventElapsedTime` with an explicit `cudaDeviceSynchronize()` before stopping host timers. Every configuration executes one discarded warm-up iteration followed by 10 measured iterations, of which the **median** is reported (median rather than mean, to reduce sensitivity to OS scheduling noise and DVFS transients). Both **kernel-only** and **end-to-end** (H2D + kernel + D2H) times are recorded for every GPU configuration.

Correctness validation. Each implementation's output is compared element-wise against the serial reference. Because floating-point summation order differs between implementations, exact equality is not expected; the acceptance criterion is maximum absolute error below 1e-4 with RMS error reported alongside.

Analytical framework. The arithmetic-intensity argument is stated quantitatively: the naive kernel performs $2K^2$ FLOPs per output pixel while loading $4K^2$ bytes of input, giving AI ≈ 0.5 FLOP/byte regardless of K. The tiled kernel loads $(T+K-1)^2/T^2$ input elements per output pixel, so for T=16, K=5 the load count falls from 25 to ≈1.56 — a reuse factor of about 16× — raising AI to ≈8 FLOP/byte and shifting the kernel rightward on the roofline toward the compute-bound regime. The measured serial fraction f (image I/O, memory allocation, filter setup, and, for GPU versions, PCIe transfer) is extracted from the profile and substituted into Amdahl's law to predict the speedup ceiling, which is then plotted against the observed plateau.

E. Hardware Requirements

TABLE II
Hardware Requirements

Component	Specification
CPU	Multi-core x86-64, ≥ 4 physical cores (8+ preferred for an informative scaling curve)
RAM	16 GB (an 8K single-precision image is ≈ 133 MB; buffers plus MPI strips need headroom)
GPU	NVIDIA, Compute Capability ≥ 6.0, ≥ 4 GB VRAM (e.g., GTX 1650 / RTX 3050 / RTX 3060); Colab T4 or a cluster node is an acceptable substitute
Storage	≈ 5 GB for the image dataset, build artefacts, and result logs
Cluster (opt., MPI)	2–4 nodes, or multiple ranks on a single node for functional demonstration

F. Software Requirements

TABLE III
Software Requirements

Component	Specification
OS	Ubuntu 22.04 LTS (native or WSL2)
Compiler	GCC 11+ or Clang 14+, OpenMP 4.5+ (`-fopenmp`)
GPU toolchain	CUDA Toolkit 12.x (`nvcc`), matching NVIDIA driver
MPI (optional)	OpenMPI 4.1+ or MPICH
Profiling	NVIDIA Nsight Compute / Nsight Systems (`ncu`, `nsys`); `nvidia-smi`
Image I/O	stb_image / stb_image_write, or OpenCV 4.x — used strictly for file decoding/encoding, never for the convolution itself
Build system	CMake 3.20+ or a hand-written Makefile
Analysis	Python 3.10+ with pandas, matplotlib, numpy
Version control	Git

G. Scope and Deliverable Size

Approximately 400–600 lines of C/C++/CUDA across all implementations, plus a Python plotting and analysis script of roughly 100 lines. The scope is deliberately bounded: the intellectual weight of the project rests on the measurement design and the analysis, not on code volume.

VI. Expected Outcomes and Applications

A. Expected Outcomes

1) A **working, validated benchmark suite** containing four (optionally five) implementations of an identical convolution algorithm, each numerically verified against the serial reference across the full sweep of image and filter sizes.
2) A **quantified OpenMP scaling curve** showing near-linear speedup at low thread counts followed by a plateau. The anticipated finding is that speedup saturates below the core count because the kernel is memory-bandwidth-bound; adding threads past the saturation point contends for the same DRAM bandwidth and yields diminishing or negative returns. Efficiency (speedup / threads) is expected to degrade monotonically, more pronounced at K=3 (low arithmetic intensity) than at K=15 (higher intensity, more compute per byte).
3) A **block-dimension sensitivity analysis** for the CUDA kernels. 8×8 blocks are expected to underperform due to only two warps per block and a poor halo-to-interior ratio in the tiled version; 16×16 is the anticipated sweet spot; 32×32 (1024 threads, the maximum block size) may be limited by register pressure and shared-memory footprint, consistent with the occupancy findings of Volkov and Demmel.
4) A **demonstrated naive-versus-tiled CUDA speedup** with an accompanying arithmetic-intensity explanation. The improvement is expected to be modest at K=3 — where the L1/L2 caches already capture most of the reuse — and to grow substantially as K increases, because the redundancy eliminated scales as K^2 while the halo overhead scales only as the tile perimeter. This size-dependence is itself a headline result.

5) An **honest accounting of transfer overhead**, presented as stacked kernel-versus-`cudaMemcpy` timing. For a single small-K convolution on one image, PCIe transfer is expected to dominate end-to-end time, meaning the GPU's apparent advantage shrinks dramatically when measured fairly. This motivates the practical conclusion that GPU offload pays off only when arithmetic work per transferred byte is sufficient — i.e., large filters, batched images, or fused multi-stage pipelines that keep data resident on the device.
6) An **Amdahl's law validation**, comparing the speedup ceiling predicted from the measured serial fraction against the observed plateau, with an explicit discussion of why the observed ceiling falls below the prediction (bandwidth saturation, synchronisation cost, and transfer latency are not captured by the model).
7) A **set of publication-quality plots** — speedup versus thread count, runtime versus filter size per implementation, runtime versus block dimension, stacked transfer-versus-kernel breakdown, and a roofline-style intensity plot — together with a written technical report interpreting them.

B. Applications

- **Real-time image and video processing:** blur, sharpen, denoise, and edge-detection filters in editing software, streaming pipelines, and camera ISPs, where per-frame latency budgets are strict.
- **Deep learning systems engineering:** the memory-hierarchy reasoning developed here is the same reasoning that underlies convolutional layer optimisation in inference engines and custom operator kernels.
- **Medical imaging:** MRI and CT reconstruction and pre-processing pipelines apply large convolution kernels to high-resolution volumes where throughput directly affects clinical turnaround.
- **Remote sensing and satellite imagery:** multi-gigapixel scenes require both GPU acceleration and distributed decomposition, exactly the combination the MPI + CUDA path demonstrates.
- **Scientific stencil computations:** finite-difference solvers in computational fluid dynamics, seismic imaging, and heat-diffusion modelling share convolution's access pattern, so the tiling and halo techniques transfer directly.
- **Educational value:** the project functions as a self-contained, reproducible case study demonstrating that on modern hardware, memory locality — rather than raw thread count — is the dominant determinant of parallel performance.

VII. Work Plan / Timeline

(Optional — include if your template requires it.)

TABLE IV
Project Timeline

Wk.	Activity	Deliverable
1	Environment setup, dataset preparation, serial baseline, correctness validator, timing harness	Working reference implementation
2	OpenMP implementation; thread-count, schedule, and collapse sweeps	OpenMP scaling data
3	Naive CUDA kernel; block-dimension sweep; transfer-time instrumentation	CUDA baseline data
4	Constant-memory and shared-memory tiled kernels; Nsight profiling	Optimised CUDA data
5	Optional MPI layer; full benchmark sweep across all image and filter sizes	Complete results.csv
6	Plotting script, Amdahl analysis, report writing, final documentation	Report, plots, source repository

References

[1] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, Atlantic City, NJ, USA, 1967, pp. 483–485.

[2] J. L. Gustafson, "Reevaluating Amdahl's law," *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988.

[3] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[4] D. B. Kirk and W.-m. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.

[5] L. Dagum and R. Menon, "OpenMP: An industry standard API for shared-memory programming," *IEEE Computational Science and Engineering*, vol. 5, no. 1, pp. 46–55, Jan.–Mar. 1998.

[6] K. Datta *et al.*, "Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures," in *Proc. ACM/IEEE Conf. Supercomputing (SC '08)*, Austin, TX, USA, 2008, pp. 1–12.

[7] P. Micikevicius, "3D finite difference computation on GPUs using CUDA," in *Proc. 2nd Workshop on General Purpose Processing on Graphics Processing Units (GPGPU-2)*, Washington, DC, USA, 2009, pp. 79–84.

[8] V. Volkov and J. W. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proc. ACM/IEEE Conf. Supercomputing (SC '08)*, Austin, TX, USA, 2008, pp. 1–11.

[9] S. Chetlur *et al.*, "cuDNN: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014.

[10] A. Lavin and S. Gray, "Fast algorithms for convolutional neural networks," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 4013–4021.

[11] NVIDIA Corporation, *CUDA C++ Programming Guide*, Version 12.x. Santa Clara, CA, USA: NVIDIA, 2024.

[12] NVIDIA Corporation, *CUDA C++ Best Practices Guide*, Version 12.x. Santa Clara, CA, USA: NVIDIA, 2024.

[13] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, Version 5.2, Nov. 2021.

[14] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*, Version 4.0, June 2021.

[15] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[16] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.